

Project Brief: Agent-Based Stock Market Simulator

ISML Course Project

Project Idea

An agent-based simulation of a small stock market. The market is managed by a central **ExchangeAgent** (the "manager"). Multiple **TraderAgents** (the "buyers" and "sellers") interact with the market, each with a unique trading strategy. The collective actions of the traders (supply and demand) will dynamically cause the price of the stocks to emerge and change over time, visualized in a live graph.

Core Agent Design

- **TraderAgent (Buyer/Seller):**
 - **Goal:** To maximize its final cash balance.
 - **Properties:** Holds cash, a portfolio of stocks (e.g., "STOCK-A": 10), and a strategy.
 - **Strategies (Personalities):**
 - **Momentum:** Buys if the price has been rising; sells if it has been falling.
 - **Value:** Buys if the price is below its personal "fair value" threshold; sells if it's high above it.
 - **Note on "Fair Value":** The "fair value" is not a global number. It is a *personal opinion* randomly assigned to each Value agent upon creation (e.g., one agent thinks a stock is worth \$42, another thinks it's worth \$61).
 - **Random:** Buys and sells randomly.
- **ExchangeAgent (The "Manager"):**
 - **Goal:** To facilitate trades fairly and provide a real-time view of the market.
 - **Jobs:**
 - Manages the "order book" (buy/sell orders).
 - Matches buy and sell orders to execute trades.
 - Updates stock prices based on supply (sell orders) and demand (buy orders).
 - Hosts the GUI, including the live graph and strategy dashboard.

Simulation Initialization

To ensure the market is active from the start, the simulation is initialized as follows:

- **Initial Stock Prices:** Each stock's starting price is randomized around a base value (e.g., 50.0 ± 10.0).

- **Initial Agent Endowments:** Every `TraderAgent` is created with a starting amount of cash (e.g., \$1000) and an initial portfolio. This portfolio is **not** randomized; each agent receives a fixed amount (e.g., 10 units) of *every* stock available in the market (e.g., if 3 stocks exist, an agent starts with 10 of STOCK-A, 10 of STOCK-B, and 10 of STOCK-C). This ensures all agents can both buy and sell from step 1.

Key Functionality & Mechanics

- **Emergent Pricing:** The core mechanic. The price is not programmed; it **emerges** from the balance of buy vs. sell orders placed by the collective agents each step.
- **GUI Interface:** A web-based interface built with MESA.
 - **Controls:** Sliders to set the number of agents for each strategy ('num_random', 'num_momentum', 'num_value') and the total number of stocks.
 - **Live Graph:** A multi-line graph showing the price history of each stock over time.
 - **Vertical (Y) Axis:** Represents the **Price** of the stocks in dollars.
 - **Horizontal (X) Axis:** Represents **Time**, measured in simulation steps (or "ticks").
 - **Strategy Dashboard:** A 3-column dashboard that replaces a simple log. It shows, for each strategy (Random, Momentum, Value):
 - **Pending Orders (Yellow):** The number of open buy and sell orders.
 - **Completed Trades (Red/Green):** The number of trades executed in the last step.

Proposed Framework

- **Language:** Python
- **Framework:** **MESA**. It provides the scheduler (for time "ticks"), the agent/model structure, a built-in web server, and a 'ChartModule' for the live graph.